

NỘI DUNG ÔN TẬP THI KẾT THÚC HỌC PHẦN

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Phần 1: Các thuật toán Machine learning cần ghi nhớ

1. Logistic Regression

- **Ứng dụng:**
 - Phân loại ung thư (Breast Cancer).
 - Phân loại rượu vang (Wine Classification).
- **Ưu điểm:**
 - Đơn giản, dễ hiểu, hoạt động tốt trên dữ liệu tuyến tính.
- **Cách sử dụng:**

```
# Python code
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

2. Random Forest

- **Ứng dụng:**
 - Phân loại nguy cơ bệnh tiểu đường (Diabetes Classification).
 - Phân loại ung thư (Breast Cancer).
- **Ưu điểm:**
 - Xử lý tốt dữ liệu phức tạp, kháng nhiễu tốt.
 - Có thể giải quyết cả bài toán phân loại và hồi quy.
- **Cách sử dụng:**

```
# Python code
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

3. K-Nearest Neighbors (KNN)

- **Ứng dụng:**
 - Phân loại chữ viết tay (Handwritten Digits).
 - Phân loại bệnh tiểu đường (Diabetes Classification).
- **Ưu điểm:**
 - Dễ hiểu, không yêu cầu huấn luyện.

- Tốt cho dữ liệu nhỏ và đơn giản.
- **Cách sử dụng:**

```
# Python code
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

4. Naive Bayes

- **Ứng dụng:**
 - Phân loại rượu vang (Wine Classification).
 - Phân loại văn bản (Text Classification).
- **Ưu điểm:**
 - Hiệu quả với dữ liệu nhỏ và độc lập.
 - Tốt cho bài toán phân loại văn bản.
- **Cách sử dụng:**

```
# Python code
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

5. Linear Regression

- **Ứng dụng:**
 - Dự đoán giá nhà (House Price Prediction).
 - Dự đoán tiền boa (Tip Prediction).
- **Ưu điểm:**
 - Đơn giản, dễ triển khai.
 - Hiệu quả với dữ liệu tuyến tính.
- **Cách sử dụng:**

```
# Python code
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

6. Decision Tree Regressor

- **Ứng dụng:**
 - Dự đoán giá nhà (House Price Prediction).
 - Dự đoán tiền boa (Tip Prediction).
- **Ưu điểm:**
 - Không yêu cầu chuẩn hóa dữ liệu.

- Dễ trực quan hóa, phù hợp dữ liệu phi tuyến.
- **Cách sử dụng:**

```
# Python code
from sklearn.tree import DecisionTreeRegressor
model = DecisionTreeRegressor()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

7. K-Means

- **Ứng dụng:**
 - Phân cụm dữ liệu Iris.
 - Phân cụm dữ liệu Wine.
- **Ưu điểm:**
 - Tốt cho dữ liệu tuyến tính và cụm đồng nhất.
- **Cách sử dụng:**

```
# Python code
from sklearn.cluster import KMeans
model = KMeans(n_clusters=3)
model.fit(X)
labels = model.predict(X)
```

8. DBSCAN

- **Ứng dụng:**
 - Phân cụm dữ liệu không tuyến tính như Wine.
- **Ưu điểm:**
 - Tốt cho dữ liệu chứa nhiễu và cụm không đồng đều.
- **Cách sử dụng:**

```
# Python code
from sklearn.cluster import DBSCAN
model = DBSCAN(eps=1.2, min_samples=5)
labels = model.fit_predict(X)
```

9. Gradient Boosting

- **Ứng dụng:**
 - Dự đoán nguy cơ bệnh tiểu đường.
- **Ưu điểm:**
 - Mạnh mẽ, phù hợp dữ liệu lớn và phức tạp.
- **Cách sử dụng:**

```
# Python code
from sklearn.ensemble import GradientBoostingClassifier
model = GradientBoostingClassifier()
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

10. Support Vector Machine (SVM)

- **Ứng dụng:**
 - Phân loại chữ viết tay (Handwritten Digits).
- **Ưu điểm:**
 - Hiệu quả với dữ liệu phức tạp và ít nhiễu.
- **Cách sử dụng:**

```
# Python code
from sklearn.svm import SVC
model = SVC()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Nội dung phần Học máy cần ghi nhớ

Bao gồm các khái niệm, kỹ thuật, và các bước triển khai mô hình mà sinh viên cần nắm vững:

1. Các bước triển khai mô hình Học máy

1. Chuẩn bị dữ liệu:

- Tải và kiểm tra dữ liệu (các giá trị thiếu, outliers, phân phối dữ liệu).

```
# Python code
from sklearn.datasets import load_breast_cancer
from sklearn.datasets import fetch_california_housing
from sklearn.datasets import load_digits
from sklearn.datasets import load_diabetes
from sklearn.datasets import load_wine
from sklearn.datasets import load_iris
import seaborn as sns
```

- Chia dữ liệu thành tập huấn luyện và kiểm tra (thường 70-30 hoặc 75-25, 80-20).

2. Tiền xử lý dữ liệu:

- **Feature Scaling:**

```
# Python code
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

- **Encoding:** Biến đổi dữ liệu dạng phân loại thành số (Label Encoding, One-Hot Encoding).

3. Huấn luyện mô hình:

- Sử dụng `fit()` để huấn luyện trên tập dữ liệu huấn luyện.
- Ví dụ:

```
# Python code
model = LogisticRegression()
model.fit(X_train, y_train)
Dự đoán và đánh giá:
```

- Sử dụng `predict()` để dự đoán.
 - Đánh giá bằng các chỉ số như Accuracy, Precision, Recall, F1-Score.
-

3. Các chỉ số đánh giá hiệu năng

- **Accuracy (Độ chính xác):**
 - Phần trăm số dự đoán đúng trên tổng số mẫu.
 - Không phù hợp khi dữ liệu không cân bằng.
- **Precision (Độ chính xác dương):**
 - Tỷ lệ giữa số mẫu dương dự đoán đúng trên tổng số mẫu được dự đoán là dương.
- **Recall (Độ nhạy):**
 - Tỷ lệ số mẫu dương dự đoán đúng trên tổng số mẫu thực sự dương.
- **F1-Score:**
 - Trung bình điều hòa giữa Precision và Recall.
- **Confusion Matrix (Ma trận nhầm lẫn):**
 - Đo lường chi tiết số lượng mẫu dự đoán đúng và sai.

```
# Python code
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

- **ROC-AUC:**
 - Đánh giá độ phân tách giữa các lớp.

```
# Python code
from sklearn.metrics import roc_auc_score
roc_auc = roc_auc_score(y_test, y_pred_prob)
print(roc_auc)
```

4. Các kỹ thuật nâng cao

- **Cross-Validation (Kiểm tra chéo):**
 - Đánh giá mô hình bằng cách chia dữ liệu thành nhiều phần nhỏ để kiểm tra.

```
# Python code
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5)
```

```
print("Cross-Validation Scores:", scores)
```

- **Grid Search (Tìm tham số tối ưu):**
 - Tìm kiếm tham số tốt nhất cho mô hình.

```
# Python code
from sklearn.model_selection import GridSearchCV
param_grid = {'n_estimators': [50, 100, 150]}
grid = GridSearchCV(RandomForestClassifier(), param_grid, cv=5)
grid.fit(X_train, y_train)
print("Best Parameters:", grid.best_params_)
```

- **Feature Importance (Tầm quan trọng của đặc trưng):**

Xác định đặc trưng quan trọng nhất trong mô hình.

```
# Python code
model = RandomForestClassifier()
model.fit(X_train, y_train)
print("Feature Importances:", model.feature_importances_)
```

5. Đoạn mã tổng hợp cần ghi nhớ

Phân loại Logistic Regression với ma trận nhầm lẫn

```
# Python code
from sklearn.metrics import classification_report,
confusion_matrix

# Huấn luyện Logistic Regression
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Đánh giá hiệu năng
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Tính ROC-AUC cho Random Forest

```
# Python code
from sklearn.metrics import roc_auc_score, roc_curve
import matplotlib.pyplot as plt

# Dự đoán xác suất
y_pred_prob = rf_model.predict_proba(X_test)[:, 1]

# Tính AUC
roc_auc = roc_auc_score(y_test, y_pred_prob)
print(f"ROC-AUC Score: {roc_auc}")

# Vẽ đường cong ROC
fpr, tpr, _ = roc_curve(y_test, y_pred_prob)
plt.plot(fpr, tpr, label=f"ROC Curve (AUC = {roc_auc:.2f})")
plt.xlabel("False Positive Rate")
```

```
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```

Phần 2: Xử lý Ngôn ngữ Tự nhiên (Natural Language Processing - NLP)

1. Các chủ đề chính cần ôn tập

- Phân tích cảm xúc (Sentiment Analysis):**
 - Phân loại văn bản thành tích cực, tiêu cực, hoặc trung lập.
 - Thư viện: VADER, TextBlob.
 - Nhận diện thực thể (NER - Named Entity Recognition):**
 - Xác định các thực thể như tên người, địa danh, tổ chức.
 - Thư viện: spaCy.
 - Phân đoạn câu và từ (Tokenization):**
 - Chia đoạn văn thành các câu hoặc từ riêng lẻ.
 - Thư viện: nltk, spaCy.
 - Phân tích cú pháp (Syntax Parsing):**
 - Xác định cấu trúc câu, mối quan hệ giữa các từ.
 - Thư viện: spaCy.
 - Dịch ngôn ngữ tự nhiên (Machine Translation):**
 - Dịch văn bản từ ngôn ngữ này sang ngôn ngữ khác.
 - Thư viện: Transformers với mô hình MarianMT.
 - Biểu diễn văn bản (Vectorization):**
 - Chuyển đổi văn bản thành vector số để sử dụng trong học máy.
 - Kỹ thuật: TF-IDF, Bag of Words.
-

2. Các công cụ và thư viện cần nắm vững

- NLTK (Natural Language Toolkit):**
 - Công cụ phổ biến để xử lý ngôn ngữ tự nhiên.
 - Chức năng: Tokenization, stemming, lemmatization.
 - spaCy:**
 - Công cụ hiện đại, hiệu quả cho NLP.
 - Chức năng: NER, tokenization, dependency parsing.
 - VADER (Valence Aware Dictionary and sEntiment Reasoner):**
 - Dùng để phân tích cảm xúc với văn bản ngắn.
 - Transformers (Hugging Face):**
 - Sử dụng các mô hình ngôn ngữ mạnh như BERT, MarianMT.
-

3. Đoạn mã Python cần ghi nhớ

Phân tích cảm xúc với VADER

```
# Python code
import nltk
from nltk.sentiment import SentimentIntensityAnalyzer

# Tải dữ liệu cần thiết cho VADER
nltk.download('vader_lexicon')

# Phân tích cảm xúc
text = "I am so happy with the service, but the delivery was delayed."
sia = SentimentIntensityAnalyzer()
scores = sia.polarity_scores(text)

# Hiển thị kết quả
print("Phân tích cảm xúc:")
print(scores)
```

Nhận diện thực thể (NER) với spaCy

```
# Python code
import spacy

# Tải mô hình ngôn ngữ
nlp = spacy.load("en_core_web_sm")

# Nhập văn bản
text = "Barack Obama was the 44th President of the United States, born in Hawaii."

# Phân tích NER
doc = nlp(text)
print("Thực thể được nhận diện:")
for ent in doc.ents:
    print(f"{ent.text} ({ent.label_})")
```

Phân đoạn câu với NLTK

```
# Python code
from nltk.tokenize import sent_tokenize

# Văn bản đầu vào
text = "Natural Language Processing is a fascinating field. It combines linguistics and computer science."

# Phân đoạn câu
sentences = sent_tokenize(text)
print("Các câu trong văn bản:")
for i, sentence in enumerate(sentences, 1):
    print(f"{i}. {sentence}")
```

Dịch ngôn ngữ với Transformers

```
# Python code
from transformers import MarianMTModel, MarianTokenizer
```



```
# Tải mô hình dịch
model_name = "Helsinki-NLP/opus-mt-en-vi"
tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianMTModel.from_pretrained(model_name)

# Dịch câu
text = "Artificial Intelligence is revolutionizing the world."
inputs = tokenizer(text, return_tensors="pt", max_length=512,
truncation=True)
translated = model.generate(**inputs)
translation = tokenizer.decode(translated[0],
skip_special_tokens=True)

# Hiển thị kết quả
print("Dịch sang tiếng Việt:")
print(translation)
```

4. Các thuật toán quan trọng trong NLP

1. TF-IDF (Term Frequency - Inverse Document Frequency):

- Chuyển văn bản thành vector, nhấn mạnh tần suất từ trong văn bản nhưng giảm trọng số các từ phổ biến.
- Ví dụ:

```
# Python code
from sklearn.feature_extraction.text import TfidfVectorizer
texts = ["I love this product!", "The service was terrible."]
vectorizer = TfidfVectorizer()
vectors = vectorizer.fit_transform(texts)
print(vectors.toarray())
```

2. Naive Bayes cho phân loại văn bản:

- Ứng dụng phổ biến để phân tích cảm xúc hoặc phân loại email spam.
- Ví dụ:

```
# Python code
from sklearn.naive_bayes import MultinomialNB
model = MultinomialNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

3. Mô hình Seq2Seq (Sequence-to-Sequence):

- Áp dụng trong dịch ngôn ngữ và tóm tắt văn bản.
 - Ví dụ: Sử dụng mô hình Transformers như MarianMT (đã trình bày ở trên).
-

5. Các bước ôn tập cụ thể

1. Thực hành từng chức năng:

- Tokenization, stemming, lemmatization với NLTK.
 - NER, dependency parsing với spaCy.
 - 2. **Hiểu và triển khai thuật toán:**
 - Vectorization (TF-IDF, Bag of Words).
 - Phân loại văn bản với Naive Bayes hoặc Logistic Regression.
 - 3. **Tìm hiểu thêm về các ứng dụng thực tế:**
 - Phân tích cảm xúc trên tập dữ liệu lớn (Ví dụ: Twitter Sentiment Analysis).
 - Dịch ngôn ngữ tự động với các thư viện hiện đại như Transformers.
-

Phần 3: Thị giác Máy tính (Computer Vision)

1. Các chủ đề chính cần ôn tập

1. **Tiền xử lý ảnh:**
 - Chuyển đổi ảnh sang định dạng xám (grayscale).
 - Resize ảnh (thay đổi kích thước ảnh).
 2. **Phát hiện cạnh:**
 - Sử dụng các phương pháp như:
 - **Canny Edge Detection.**
 - **Sobel Edge Detection.**
 3. **Làm mịn ảnh và giảm nhiễu:**
 - Gaussian Blur.
 - Median Blur.
 4. **Phát hiện đường và hình dạng:**
 - Phát hiện đường thẳng bằng **Hough Transform.**
 - Phát hiện hình tròn bằng **Hough Circle Transform.**
 5. **Hiển thị và lưu kết quả:**
 - Hiển thị ảnh với Matplotlib.
 - Lưu ảnh sau khi xử lý với OpenCV.
-

2. Các công cụ và thư viện cần nắm vững

1. **OpenCV (Open Source Computer Vision Library):**
 - Thư viện chính để xử lý ảnh và video.
 - Hỗ trợ các thao tác cơ bản như đọc, ghi, hiển thị và xử lý ảnh.
 2. **Matplotlib:**
 - Thư viện dùng để hiển thị kết quả xử lý ảnh.
-

3. Đoạn mã Python cần ghi nhớ

Chuyển đổi ảnh sang ảnh xám và resize

```
# Python code
import cv2

# Đọc ảnh
image = cv2.imread("road.jpg")

# Chuyển ảnh sang ảnh xám
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Resize ảnh
resized = cv2.resize(gray, (128, 128))

# Lưu kết quả
cv2.imwrite("resized_road.jpg", resized)
```

Phát hiện cạnh bằng Canny

```
# Python code
# Áp dụng phát hiện cạnh Canny
edges = cv2.Canny(gray, threshold1=80, threshold2=200)

# Lưu ảnh cạnh
cv2.imwrite("canny_road.jpg", edges)
```

Phát hiện cạnh bằng Sobel

```
# Python code
# Phát hiện cạnh Sobel
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3) # Cạnh ngang
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3) # Cạnh dọc

# Lưu ảnh cạnh Sobel
cv2.imwrite("sobel_x_road.jpg", sobel_x)
cv2.imwrite("sobel_y_road.jpg", sobel_y)
```

Làm mịn ảnh bằng Gaussian Blur và Median Blur

```
# Python code
# Gaussian Blur
gaussian_blur = cv2.GaussianBlur(gray, (5, 5), 0)

# Median Blur
median_blur = cv2.medianBlur(gray, 5)

# Lưu ảnh làm mịn
cv2.imwrite("gaussian_blur.jpg", gaussian_blur)
cv2.imwrite("median_blur.jpg", median_blur)
```

Phát hiện đường thẳng bằng Hough Transform

```
# Python code
# Phát hiện cạnh Canny
edges = cv2.Canny(gray, 80, 200)

# Phát hiện đường thẳng
lines = cv2.HoughLinesP(edges, rho=1, theta=np.pi/180, threshold=100,
minLineLength=50, maxLineGap=10)

# Vẽ đường thẳng lên ảnh gốc
output_image = image.copy()
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(output_image, (x1, y1), (x2, y2), (0, 255, 0), 2)

# Lưu ảnh kết quả
cv2.imwrite("hough_lines_road.jpg", output_image)
```

4. Phân tích hiệu quả và ứng dụng

1. Hiệu quả của các thuật toán:

- **Canny Edge Detection:**
 - Hiệu quả khi cần phát hiện các cạnh rõ ràng.
 - Yêu cầu xác định ngưỡng thấp và cao phù hợp.
- **Sobel Edge Detection:**
 - Phù hợp để phát hiện cạnh theo hướng cụ thể (ngang, dọc).
- **Hough Transform:**
 - Tốt cho việc phát hiện các đường thẳng hoặc hình dạng như hình tròn.

2. Ứng dụng thực tế:

- **Phát hiện làn đường giao thông:** Dùng Canny và Hough Transform.
 - **Nhận dạng vật thể:** Kết hợp Sobel và Gaussian Blur để xử lý dữ liệu đầu vào.
-

5. Các bước ôn tập cụ thể

1. Thực hành từng thuật toán:

- Tự triển khai các thuật toán trên các ảnh mẫu như `road.jpg`, `flower.jpg`, `landscape.jpg`.

2. So sánh kết quả:

- Thử nghiệm các giá trị tham số khác nhau (ví dụ: ngưỡng trong Canny, kích thước kernel trong Gaussian Blur).

3. Hiểu lý thuyết:

- Đọc hiểu cách hoạt động của mỗi thuật toán, ưu nhược điểm và khi nào nên sử dụng.